강의 · Claude 업무 시스템

나만의 AI 업무 시스템

SSR+ 네 단계로 만드는 Claude 대시보드

스킬에서 시작해 세컨드 브레인과 루틴을 거쳐 익스텐션까지 세운다

AI 성능이 아니라 일하는 환경이 생산성을 정한다.
영상 속 시스템을 Claude Code로 직접 구현하는 순서를 따라간다.

대시보드는 결과이고 바닥은 네 단계 세팅이다

이 강의 전체를 관통하는 세 가지 주장

1

화면보다 바닥을 먼저 깐다

스킬, 세컨드 브레인, 루틴, 익스텐션을 깔아야 AI가 맥락을 갖고 움직인다.

2

모든 단계는 하나의 작업 폴더 위에서 돈다

옵시디언 볼트를 Claude Code 프로젝트로 겸용하면 지식과 자동화가 한곳에 모인다.

3

도구를 배우지 않고 만들라고 시킨다

탐색기, 동기화, 커넥터 찾기, 대시보드까지 프롬프트 한 장으로 맡긴다.

목표: 영상 속 시스템을 오늘 내 컴퓨터에서 똑같이 재현한다

일이 줄지 않는 이유는 작업 환경의 부재다

SSR+의 네 용어를 먼저 정의한다 — S 스킬, S 세컨드 브레인, R 루틴, + 익스텐션

스킬

업무 단축키

반복 업무의 절차와 참고 자료를 묶은 폴더다. 맥락이 맞으면 저절로 발동한다.

예) 견적서를 HWP로

세컨드 브레인

AI가 읽는 지식 지도

AGENTS.md와 길잡이 파일로 AI가 자료를 스스로 찾게 한다.

예) 촬영 원본 찾기

루틴

예약된 단축키

정해진 시간에 스킬이나 프롬프트를 자동으로 실행한다.

예) 매일 아침 브리핑

익스텐션

외부 도구 연결

메일, 캘린더, 브라우저, 공공데이터를 AI가 다루게 연결한다.

예) Gmail 커넥터

출발점: 같은 작업을 두 번 이상 시켰다면 스킬로 만든다

대시보드 한 화면에 지식과 운영 현황이 모인다

영상 속 대시보드 구성 — 칸마다 아래 꼬리표가 그 칸을 받치는 단계다

채널과 목표

유튜브와 뉴스레터 현황
이번 달 목표와 캘린더

익스텐션 + 루틴

세컨드 브레인

지식 연결을 보고 검색한다



길잡이 파일이 받친다

메일함

답장이 필요한
메일을 요약한다

익스텐션

스킬

누르면 실행되는 단축키
모델과 프롬프트 선택

스킬

루틴

정해진 시간에 자동 실행
마지막 실행 시각 표시

루틴

네 단계가 쌓인 결과를 대시보드가 보여 준다

작업 폴더(볼트) 하나 위에 네 단계가 올라가고 대시보드가 그것을 모아 보여 준다



단계마다 초급에서 고급으로 오르는 길이 있다

영상이 제시한 수준별 세팅 방법을 한 표로 정리했다

단계	초급	중급	고급
스킬	스킬 크리에이터에게 말해서 만든다	스킬 폴더에 참고 자료를 넣는다	스킬을 고치는 스킬을 둔다
세컨드 브레인	AGENTS.md에 지침을 적는다	주제별 길잡이 파일을 만든다	나만의 탐색기를 만든다
루틴	데스크톱 앱 예약 작업을 쓴다	상시 켜진 PC에 Hermes를 둔다	자료와 실행을 모두 클라우드에 둔다
익스텐션	기본 커넥터를 클릭해 연결한다	MCP, CLI, API 순으로 찾는다	필요한 도구를 직접 만든다

루틴의 고급 단계는 영상 시점 기준으로 아직 완성되지 않았다

01

LEVEL 1 · 스킬

스킬 — 반복 업무를 단축키로

같은 작업을 두 번 시켰다면 스킬로 만든다

첫 스킬은 스킬 크리에이터에게 말로 주문한다

데스크톱 앱 사용자 지정의 스킬 메뉴에 기본 스킬 skill-creator가 들어 있다

[입력] 프롬프트 · Claude Code

```
skill-creator로 반복 업무를 스킬로 만들고 싶다.
먼저 나에게 질문 5개만 해서 후보를 뽑아라.
상위 3개를 .claude/skills/ 아래에 만들어라.
description은 '언제 쓰는지'가 한 문장으로
드러나게 써라.
```

[출력] Claude가 생성 · 스킬 폴더 3개

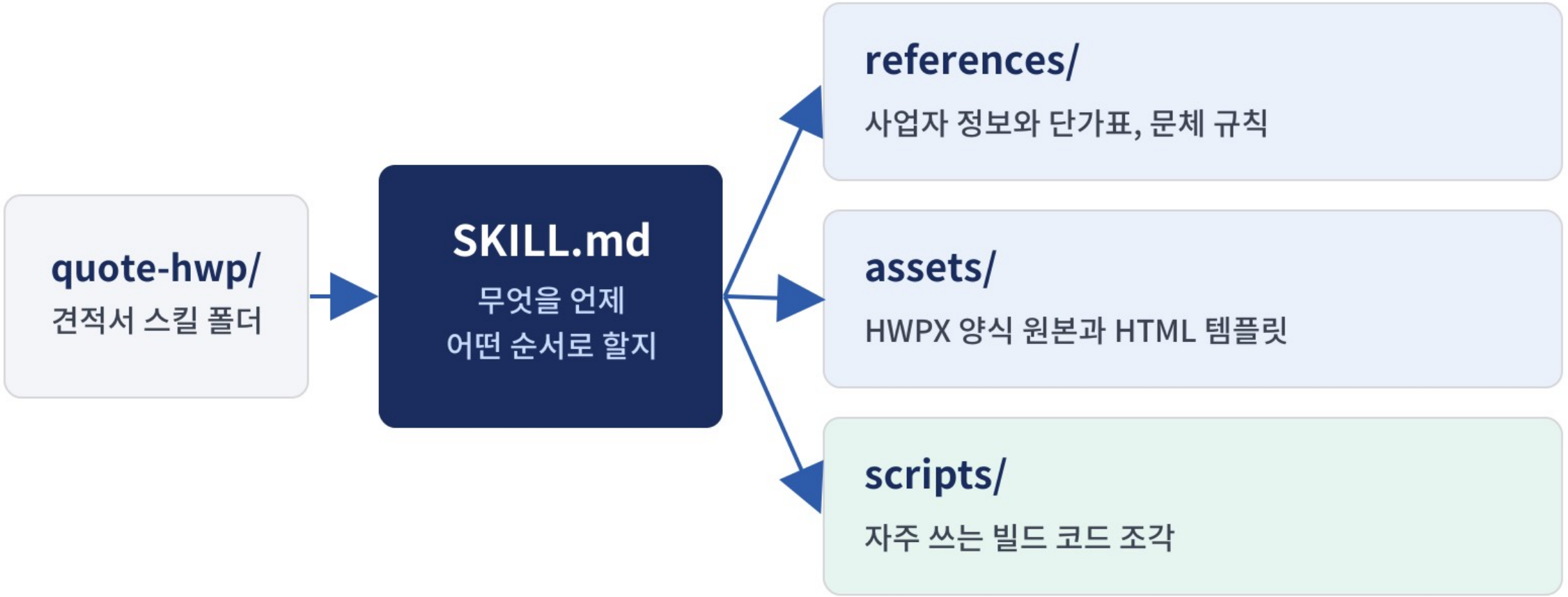
```
.claude/skills/
├──── quote-hwp/SKILL.md
├──── weekly-report/SKILL.md
└──── deck-outline/SKILL.md
```

에피소드 · PC 점검 스킬

- 트위터에서 본 PC 점검 프롬프트를 붙여 넣고 스킬로 만들어 달라고 했다
- 이후 'PC 점검해 줘' 한마디로 스킬이 실행됐다
- 확인: 이름을 부르지 않아도 발동하는가

참고 자료를 넣은 스킬은 결과가 일정하다

SKILL.md는 지시만 하고 세부 규칙과 양식은 하위 폴더에 둔다 — 코드 조각을 넣으면 더 빠르다



스킬은 낡으므로 고치는 장치를 함께 둔다

회사 양식이 바뀌면 예전 스킬은 틀린 결과를 낸다

[입력] 프롬프트 · 방식 A · 스킬마다 업데이트 절차

```
모든 SKILL.md 끝에 '업데이트 절차'를
추가해라. 내가 결과를 고치면 그 차이를
references/changelog.md에 기록하고,
같은 수정이 두 번 나오면
SKILL.md 규칙을 고치자고 제안한다.
```

[입력] 프롬프트 · 방식 B · 스킬을 고치는 스킬

```
skill-reviewer 스킬을 만들어라.
스킬이 발동된 최근 대화를 읽고, 반복해서
고친 부분과 발동이 빗나간 경우, 낡은
양식을 찾아 스킬별 개선안을 표로 낸다.
고치기 전에 반드시 나에게 확인받는다.
```

[피드백] 이렇게 확인한다

- 개선안 표에서 승인한 것만 반영하고, git 커밋으로 이전 버전을 남긴다

02

LEVEL 2 · 세컨드 브레인

세컨드 브레인 — AI가 자료를 찾게

폴더 정리 대신 길잡이 파일을 정리한다

AGENTS.md는 AI가 먼저 읽는 업무 지침서다

CLAUDE.md나 AGENTS.md라는 이름의 파일은 세션을 열 때마다 읽힌다

[입력] 프롬프트 · Claude Code

```
나와 인터뷰하듯 질문해서 AGENTS.md를 써라.
1) 나는 누구이고 어떤 일을 하는가
2) 항상 지킬 규칙 (언어, 파일명, 용어 표기)
3) 업무 영역별로 먼저 볼 길잡이 파일
4) 하지 말 것
200줄을 넘기지 말고 세부는 길잡이로 넘겨라.
```

[출력] 결과 · AGENTS.md (일부)

```
## 항상 지킬 규칙
- 답은 한국어로, 파일명은 영어로 쓴다
## 업무 영역별 길잡이
- 강의와 덱: _index/content.md
```

CLAUDE.md와의 관계

- Claude Code는 CLAUDE.md를 자동으로 읽는다
- CLAUDE.md에 '@AGENTS.md' 한 줄을 두면 AGENTS.md를 함께 불러온다
- 다른 에이전트 도구도 AGENTS.md를 읽어 지침을 공유한다

길잡이 파일은 링크로 자료의 위치를 알린다

AGENTS.md가 대문, _index/ 아래 파일이 주제별 안내 페이지 역할을 한다



길잡이가 있으면 모호한 요청도 바로 닿는다

에이전트가 파일을 대신 찾는 시대에는 폴더 정리에 집착할 필요가 없다

[입력] 프롬프트 · Claude Code

```
볼트 전체를 훑어 _index/ 아래에
주제별 길잡이 파일을 만들어라.
각 파일은 진행 중, 자주 찾는 원본,
하위 길잡이의 세 절로 쓰고 위키링크를 쓴다.
AGENTS.md의 길잡이 절도 맞춰 고쳐라.
길잡이에 없는 파일은 orphans.md로 낸다.
```

[출력] 결과 · _index/content.md

```
# content 길잡이
## 진행 중
- [[01_PROJECTS/lecture-a/README]]
## 자주 찾는 원본
- 촬영 원본: D:/video/raw/ (최신이 위)
## 하위 길잡이
- [[_index/content-video]]
```

에피소드 · 촬영 원본 찾기

- 파인더 검색은 엉뚱한 파일부터 보여 줬지만 대시보드 검색은 직전 촬영본을 바로 찾았다

탐색기는 내 지식을 내 방식으로 보여 준다

옵시디언이나 노션 사용법을 배우는 대신 나에게 맞는 탐색기를 만들라고 시킨다

[입력] 프롬프트 · Claude Code

```
dashboard/explorer/에 로컬 웹앱을 만들어라.
AGENTS.md를 중심으로 길잡이와 위키링크를
그래프로 그리고 계층 뷰로도 바꾸게 해라.
파일을 누르면 오른쪽에 미리 보기를 띄운다.
검색창은 파일명과 본문을 모두 찾는다.
npm run explorer 하나로 뜨게 해라.
```

옵시디언을 이미 쓴다면

- 그래프 뷰와 검색은 옵시디언이 이미 한다
- 탐색기는 대시보드 가운데 칸에 넣을 때만 만든다



03

LEVEL 3 · 루틴

루틴 — 정해진 시간에 저절로

스킬이 단축키라면 루틴은 '오전 9시에 그 단축키를 눌러라'이다

첫 루틴은 데스크톱 앱 예약 작업으로 만든다

데스크톱 앱 Code 탭의 예약 메뉴에서 새 작업을 만들고 일정을 정한다

[입력] 프롬프트 · 예약 작업 (매일 08:50)

```
Gmail, Google Calendar, 볼트의 01_PROJECTS를
읽고 오늘 브리핑을 만들어라.
1) 오늘 일정 2) 답장이 필요한 메일 상위 5개
3) 마감이 임박한 프로젝트
결과는 02_OPERATIONS/briefings/에 md로 쓴다.
끝나면 dashboard/data/runs/<기기이름>.jsonl에
실행 결과를 한 줄 남긴다.
```

[출력] 결과 · runs/mac.jsonl (한 줄)

```
{"routine":"morning-briefing","device":"mac",
 "at":"2026-11-02T08:52","result":"완료"}
```

에피소드 · 루틴이 하나뿐

- 영상 속 예약 목록에는 루틴이 하나만 보였다
- 데스크톱 예약 작업은 컴퓨터와 앱이 켜져 있을 때만 돈다
- 그래서 나머지 루틴은 꺼지지 않는 컴퓨터로 옮겼다

꺼지지 않는 루틴에는 상시 실행기가 필요하다

상시 실행기에 Hermes Agent 같은 상주형 에이전트를 설치해 루틴을 맡긴다

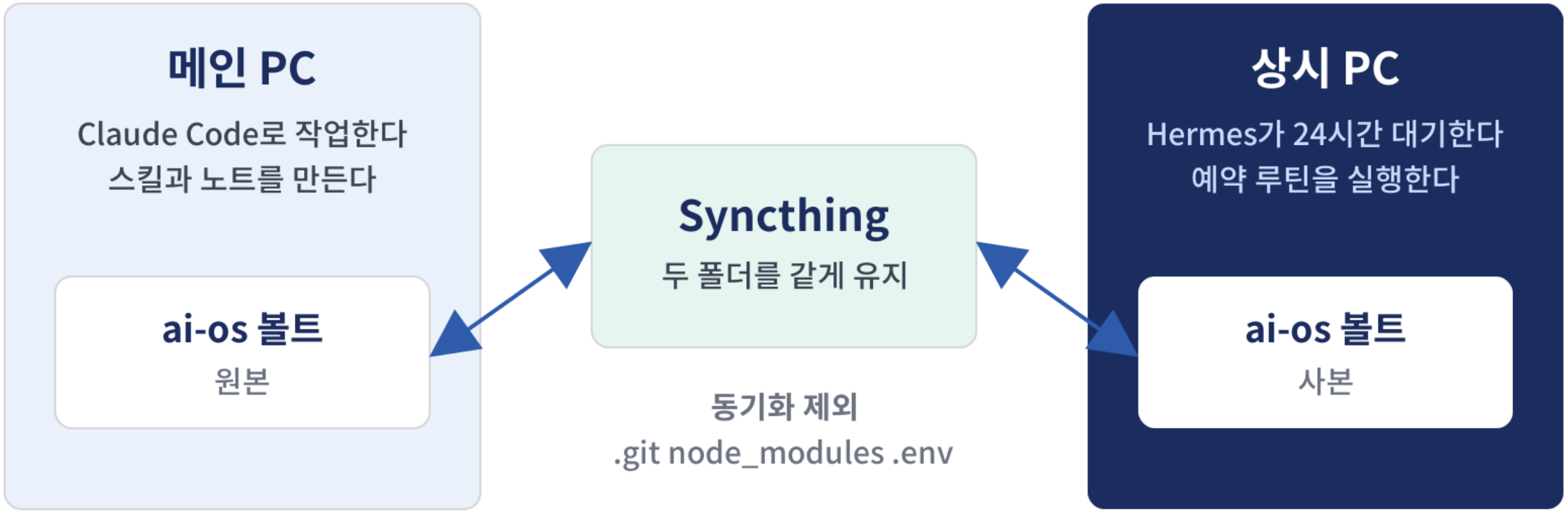
선택지	장점	단점
남는 PC에 Hermes 설치	비용이 없고 자료가 집에 있다	정전과 재부팅을 관리해야 한다
클라우드 서버(VPS)에 Hermes 설치	24시간 안정적으로 돈다	월 비용이 들고 자료가 외부에 있다
Claude 클라우드 예약 작업	설치할 것이 없다	로컬 파일을 못 읽어 볼트를 git에 올려야 한다

영상 화자는 집에 남는 윈도우 PC에 Hermes를 두고 루틴을 돌린다

자막의 'LMS'는 Hermes를 잘못 받아 적은 것으로 보인다

Syncthing이 볼트를 두 컴퓨터에 똑같이 유지한다

메인 PC에서 만든 스킬과 노트가 상시 PC로 자동으로 넘어간다



메인 PC를 꺼 두어도 루틴은 상시 PC에서 돈다

동기화 설정도 AI에게 맡기고 결과만 확인한다

메인 PC에서는 Claude Code가, 상시 PC에서는 Hermes가 설정을 진행한다

[입력] 프롬프트 · Claude Code → 이어서 Hermes

```
Syncthing을 설치하고 ai-os 폴더를
보내기와 받기가 모두 되는 공유로 설정해라.
.git/, node_modules/, .env는
.stignore로 제외한다.
상시 PC가 받을 기기 ID를 알려 주고
연결 확인 방법을 안내해라.
```

[피드백] 이렇게 확인한다

- 메인 PC에서 메모 하나를 만든다
- 1분 안에 상시 PC에 생기는지 본다
- API 키가 든 .env는 넘어가지 않아야 한다

고급 · 처음부터 클라우드로

- 자료와 Claude Code를 모두 클라우드에 두는 방식은 아직 완성 단계가 아니다
- 지금은 모닝 브리핑 하나만 옮겨 시험해 보는 정도가 현실적이다

04

LEVEL 4 · 익스텐션

익스텐션 — 외부 도구를 손발로

기본 커넥터로 시작하고, 없으면 찾고, 그래도 없으면 만든다

연결 방법은 쉬운 것부터 어려운 순서로 찾는다

커넥터를 클릭하고, 없으면 MCP, CLI, API 순으로 내려가고, 마지막에 직접 만든다



커넥터 찾기 자체를 스킬로 만들어 둔다

예) 건축물대장을 연결하려면 건축물대장 MCP부터 검색하고 README를 읽힌다

[입력] 프롬프트 · Claude Code

```
connector-finder 스킬을 만들어라.
내가 'X를 연결하고 싶다'고 하면
1) X용 MCP 서버를 찾아 README를 요약하고
2) 없으면 CLI, 그다음 API를 찾는다.
3) 후보마다 인증 방식, 비용, 신뢰도를 표로 낸다.
4) 내가 고르면 설치하고 연결을 시험한다.
   키 입력은 내가 직접 한다.
```

[출력] 결과 · 후보 비교표

후보	종류	인증	비용	신뢰도
...	MCP	API 키	무료	공식 배포

초급 · 기본 커넥터부터

- Claude 앱 설정의 커넥터 메뉴에서 Gmail과 Google Drive를 클릭해 연결한다
- 연결 뒤에는 '내 Gmail 확인해 줘'처럼 바로 시킨다
- 대부분 해외 서비스라 국내 업무용은 따로 찾는다

맞는 도구가 없으면 직접 만들어 쓴다

만들고 싶은 기능을 말하면 에이전트가 준비물과 절차를 안내한다

[입력] 프롬프트 · Claude Code

```
내 볼트와 최근 한 달 작업 기록을 보고
내가 반복해서 불편해한 지점을 찾아라.
직접 만들면 좋을 익스텐션 후보 5개를 내고
후보마다 만드는 데 걸릴 시간과
필요한 준비물을 적어라.
```

후보 예

- 덱과 그림을 썸네일로 모아 보는 라이브러리
- 강의 영상 원본을 날짜순으로 보는 라이브러리

에피소드 · 마크다운 뷰어

- 노션과 옵시디언 사용법이 불편해 직접 뷰어를 만들었다
- 한 창에 파일 두세 개를 나란히 띄워 본다
- 대시보드의 콘텐츠 라이브러리도 같은 방식으로 만들었다

05

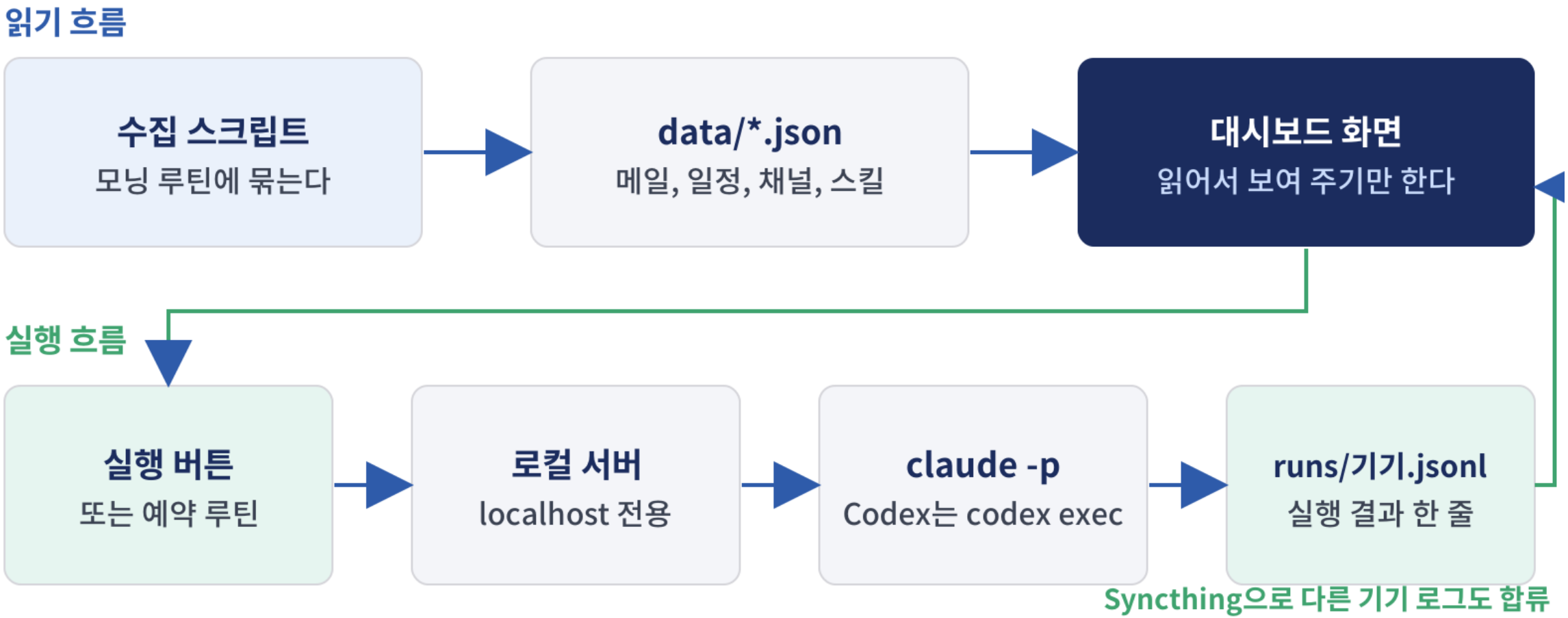
LEVEL 5 · 대시보드

대시보드 — 전체를 한 화면에

네 단계가 깔리면 마지막에 화면을 씌운다

대시보드는 읽기만 하고 실행은 로컬 서버가 맡는다

실행 결과는 기기별 로그(runs/)에 쌓이고 Syncthing으로 다른 기기와 합쳐진다



화면보다 데이터 계약을 먼저 정한다

화면부터 만들면 숫자가 가짜로 채워진다 — 순서는 데이터 계약, 수집, 화면이다

[입력] 프롬프트 · Claude Code (볼트 루트)

```
dashboard/data/에 다음 JSON 스키마와 예시를 만들어라.
- channels.json: 채널별 현재값, 전일값,
  수집 시각, 갱신 기한(24시간)
- goals.json: 기간, D-day, 항목별 진행률
- calendar.json: 월간 일정과 주차별 일정 수
- mail.json: 최근 20건과 확인 시각
- skills.json: .claude/skills와 Codex 스킬 폴더 스캔
  (에이전트, 이름, 설명, 상태, 최근 실행 시각)
- routines.json: 루틴마다 기기(mac/windows), 일정
- brain.json: _index 영역별 파일 수와 파일 목록
루틴은 끝날 때마다 runs/<기기이름>.jsonl에 한 줄을 남긴다.
스키마는 data/SCHEMA.md에 표로 정리해라.
```

[피드백] 이 단계에서 고친다

- SCHEMA.md 표를 읽고 빠진 칸을 지금 채운다
- 화면을 만든 뒤 고치면 계약, 수집, 화면 세 곳을 다시 손봐야 한다

수집 스크립트가 계약대로 실제 값을 채운다

유튜브는 YouTube Data API로, 메일과 캘린더는 연결된 커넥터로 읽는다

[입력] 프롬프트 · Claude Code (볼트 루트)

```
1단계의 JSON을 채우는 수집 스크립트를
dashboard/collect/에 만들어라.
채널 값은 날마다 data/history/에 스냅샷으로 남기고
전일 대비를 계산한다.
skills.json과 brain.json은 폴더를 스캔해 다시 만든다.
모닝 루틴에 묶고, 끝나면 runs/에 한 줄을 남긴다.
API 키는 .env에서만 읽는다.
```

[출력] 결과 · data/channels.json (일부)

```
{"youtube":{"now":338371,"prev":328325,
 "collected_at":"2026-11-02T08:50","stale_after_h":24}}
```

왜 스냅샷을 남기나

- API는 현재값만 준다
- 어제 값을 저장해 두어야 전일 대비를 계산한다
- 수집 시각을 남겨야 데이터가 멈춘 것을 알아챈다

화면은 마지막에 씌우고 모양은 자유롭게 한다

영상과 똑같을 필요는 없다 — 3열 배치와 포인트 색 하나만 정한다

[입력] 프롬프트 · Claude Code (볼트 루트)

```
dashboard/에 로컬 웹앱을 만들어라. data/의 JSON과 runs/만 읽는다. 3열로 배치하고 포인트 색은 하나만 쓴다.
- 왼쪽: 채널 카드(현재값, 전일 대비, 수집 시각. 기한을 넘기면 '갱신 필요' 표시),
  목표와 D-day, 월간 캘린더와 선택한 날짜의 일정, 최근 파일, 다른 대시보드 링크
- 가운데: 세컨드 브레인. 영역별 파일 수와 탐색기(검색, 미리 보기)
- 오른쪽: 메일(안 읽은 수, 최근 20건, 확인 시각), 스킬(검색, 에이전트 표시, 상태, 실행 버튼),
  루틴(runs/의 모든 기기 로그를 모아 기기 이름, 일정, 최근 결과)
실행 버튼은 로컬 서버가 claude -p (Codex 스킬은 codex exec)로 호출하고
결과를 runs/<기기이름>.jsonl에 남긴다.
실행은 localhost에서만 받고 외부 접속은 막아라. npm run dashboard 하나로 뜨게 해라.
```

[출력] 결과 · 브라우저

```
http://localhost:<포트>
3열 대시보드가 열린다
```

[피드백]

- 버튼으로 스킬을 실행해 볼트에 산출 파일이 생기는지 본다

4주면 영상 속 시스템을 그대로 재현할 수 있다

주마다 한 층씩 쌓고 완료 기준으로 점검한다

1주 스킬	2주 세컨드 브레인	3주 루틴과 익스텐션	4주 대시보드
볼트를 정비하고 스킬 3개를 만든다	AGENTS.md와 길잡이 파일을 만든다	브리핑 루틴과 Gmail, 상시 PC를 연결한다	대시보드와 skill-reviewer를 만든다
완료 기준 이름을 부르지 않아도 스킬이 발동한다	완료 기준 모호한 요청으로 파일을 바로 찾는다	완료 기준 컴퓨터를 꺼도 브리핑이 생성된다	완료 기준 화면에서 스킬을 실행한다

디자인보다 세 가지 동작을 점검한다

대시보드가 완성됐는지는 모양이 아니라 아래 세 가지로 판단한다

1

숫자가 살아 있는가

루틴이 돈 뒤 채널과 메일 값이 바뀌고, 수집 시각이 화면에 보인다.

2

스킬 버튼이 실제로 도는가

버튼을 누르면 볼트에 산출 파일이 생기고 카드에 완료가 표시된다.

3

꺼 둔 컴퓨터의 루틴도 보이는가

상시 PC에서 돈 루틴의 결과가 메인 PC 대시보드에 나타난다.

세 번째가 안 되면 runs/ 폴더가 Syncthing 동기화에서 빠져 있지 않은지 본다

내 업무에 옮기기 전에 네 질문에 답한다

각 질문의 답이 곧 첫 번째 스킬, 길잡이, 루틴, 커넥터 후보가 된다

Q1	지난 한 달 동안 두 번 이상 시킨 업무는 무엇인가?	**스킬**
Q2	AI가 자주 못 찾는 자료는 어디에 흩어져 있는가?	**세컨드 브레인**
Q3	컴퓨터를 꺼 둔 시간에도 돌아야 하는 일은 무엇인가?	**루틴**
Q4	기본 커넥터에 없지만 꼭 연결할 국내 서비스는 무엇인가?	**익스텐션**

바닥부터 쌓고 화면은 마지막에 씌운다

이 강의의 세 가지 결론

1

바닥부터 쌓는다

스킬, 세컨드 브레인, 루틴, 익스텐션을 차례로 깔고 대시보드는 마지막에 만든다.

2

하나의 볼트에 모은다

AGENTS.md를 대문으로, 길잡이 파일을 안내 페이지로 두면 AI가 스스로 자료를 찾는다.

3

만들라고 시킨다

탐색기, 동기화, 커넥터 찾기, 대시보드까지 모두 프롬프트로 Claude Code에 맡긴다.

과제: 반복 업무 하나를 스킬로 만들고 브리핑 루틴을 일주일 돌려 본다